INTERNSHIP PROJECT REPORT

SMART AUTOMATION SYSTEM
USING ARDUINO AND LDR SENSOR

Embedded Systems & IoT Internship

Submitted by :

Saurav Joshi

B.Tech – Electronics and Communication Engineering

Project Simulation Platform: Tinkercad Circuits

Programming: Arduino C/C++

CERTIFICATE

This is to certify that Saurav Joshi has completed the project titled “Smart Automation System Using Arduino and LDR Sensor” as part of Task 3 of the Embedded Systems & IoT internship.

The project demonstrates the use of an Arduino UNO, LDR sensor, LED and conditional automation logic to monitor light intensity and control an output automatically. The work was developed and tested using Tinkercad Circuits simulation.The project has been prepared as a practical demonstration of sensor interfacing, embedded programming, decision-making and real-time monitoring.

DECLARATION

I hereby declare that the project report titled “Smart Automation System Using Arduino and LDR Sensor” is prepared by me as part of my Embedded Systems & IoT internship Task 3.

The project work is based on my practical implementation and simulation using Arduino UNO, an LDR sensor and an LED. The circuit, program logic, observations and explanations presented in this report have been prepared for academic and internship purposes.I have made reasonable efforts to understand the working of each component and the automation logic used in the project.

Saurav Joshi

Date: _____

TABLE OF CONTENTS

1. INTRODUCTION

Automation is an important concept in embedded systems and IoT. It allows a system to sense its environment, process information and perform an action without continuous human intervention. Sensor-based automation is widely used in smart homes, offices, streets, industries and energy-management systems.

This project presents a basic Smart Automation System using an Arduino UNO and an LDR (Light Dependent Resistor) sensor. The LDR is used to detect changes in surrounding light intensity. The Arduino reads the sensor value through its analog input and makes an automatic decision using IF/ELSE logic.

An LED is used as the output device. When the simulated environment is considered dark, the LED is switched ON. When sufficient light is detected, the LED is switched OFF. The Serial Monitor provides live sensor values and system status, making the project useful for both automation and monitoring.

1.1 Background

The project is an upgraded version of the Smart Lighting System developed in Task 2. Task 3 extends the earlier sensor prototype by adding clearer automation logic, status reporting and real-time monitoring through the Serial Monitor.

1.2 Purpose of the Project

The purpose is to demonstrate how a simple sensor can be integrated with a microcontroller to create an automatic decision-making system. The project also builds practical understanding of analog sensor interfacing and embedded C/C++ programming.

2. PROJECT OVERVIEW AND OBJECTIVES

The Smart Automation System consists of an Arduino UNO, LDR sensor, resistors and LED. The LDR generates an analog signal that changes according to light intensity. Arduino continuously reads this signal and compares it with a predefined threshold value of 500.

2.1 Main Objective

The main objective is to convert a basic light-sensing circuit into an automated system that can detect a light condition, make a decision and control an LED accordingly.

2.2 Specific Objectives

- Interface an LDR sensor with Arduino UNO.
- Read analog sensor data through analog pin A0.
- Use a threshold value of 500 for decision making.
- Automatically control an LED through digital pin 13.
- Display sensor values on the Serial Monitor.
- Display the detected condition and LED status.
- Understand basic automation and embedded-system logic.

2.3 Expected Outcome

After successful implementation, the system should respond automatically to changes in the LDR reading. A reading above 500 is treated as a dark condition and turns the LED ON, while a reading of 500 or below is treated as a bright condition and turns the LED OFF.

2.4 Project Features

- Automatic operation
- Simple sensor-based control
- Real-time Serial Monitor feedback
- Low-cost components
- Easy-to-understand programming
- Suitable for simulation and learning

3. COMPONENTS AND SOFTWARE REQUIREMENTS

3.1 Hardware Components

Component	Quantity	Purpose
Arduino UNO	1	Main microcontroller and decision-making unit
LDR Sensor	1	Detects surrounding light intensity
10k Ohm Resistor	1	Forms the LDR voltage-divider circuit
LED	1	Indicates the system output
220 Ohm Resistor	1	Limits current through the LED
Breadboard	1	Provides convenient circuit assembly
Jumper Wires	As required	Electrical connections
USB Cable	1	Programming/power connection

3.2 Software Requirements

- Tinkercad Circuits – for circuit construction and simulation.
- Arduino IDE – for writing and testing Arduino code.
- Embedded C/C++ – programming language used for the microcontroller.

3.3 Arduino UNO

Arduino UNO is the central controller of the project. It reads the analog value from the LDR through A0, executes the programmed IF/ELSE condition and controls the LED through digital pin 13.

3.4 LDR Sensor

An LDR is a light-sensitive resistor whose resistance changes with light intensity. In the project, it is connected as part of a voltage divider so that Arduino can read a varying analog voltage.

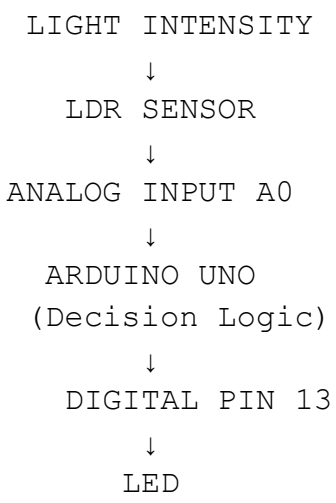
3.5 LED

The LED acts as a visual output indicator. Its state represents the decision made by the Arduino based on the LDR reading.

4. SYSTEM DESIGN AND CIRCUIT DESCRIPTION

The system follows a simple sensor–controller–output architecture. The LDR provides the input, Arduino processes the input and the LED provides the output.

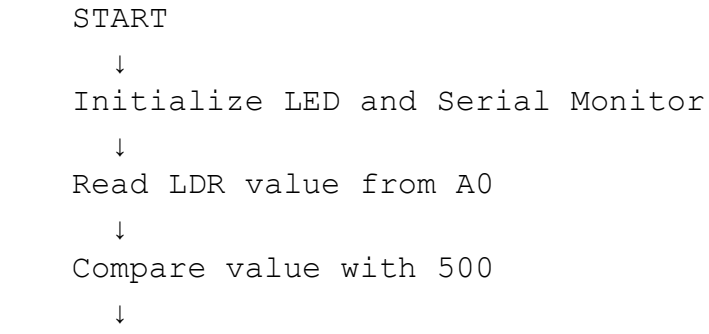
4.1 Block Diagram



4.2 Circuit Connections

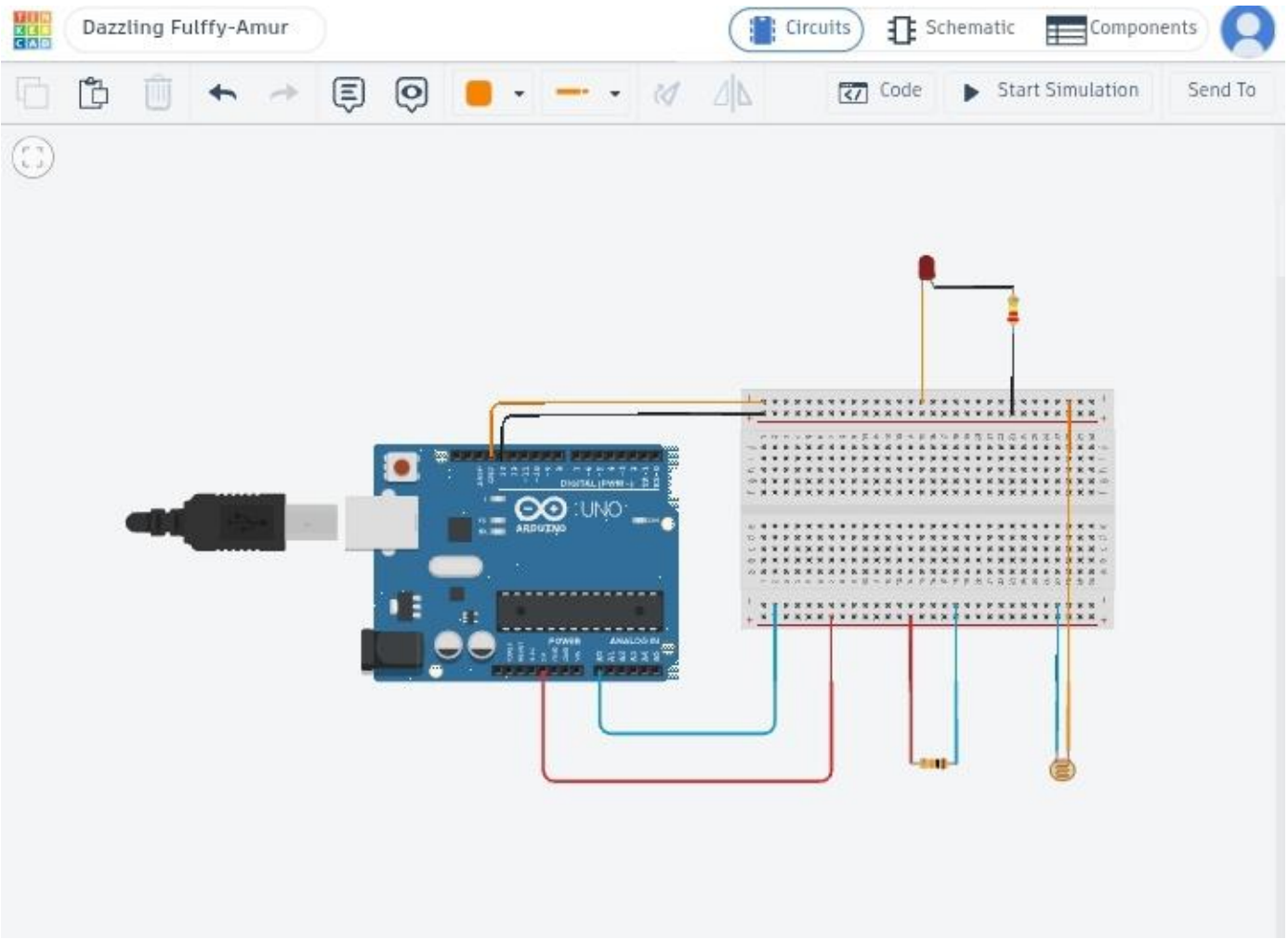
- LDR voltage-divider output is connected to Arduino analog pin A0.
- A 10k Ohm resistor is used with the LDR in the sensor circuit.
- LED is connected to Arduino digital pin 13 through a 220 Ohm resistor.
- The circuit uses common Arduino ground and appropriate power connections.
- The circuit can be assembled and tested in Tinkercad Circuits.

4.3 System Flow



```
If value > 500 → DARK → LED ON
   ↓
Else → BRIGHT → LED OFF
   ↓
Display value and status
   ↓
Wait 1 second
   ↓
Repeat
```

4.4 Circuit Diagram



The final circuit diagram should be included in the submitted report as an image exported or captured from the Tinkercad simulation. Recommended file name: Circuit_Diagram.jpg.

5. **WORKING PRINCIPLE AND AUTOMATION LOGIC**

The working of the system begins when the Arduino starts and initializes the LED output and Serial Monitor. The LDR continuously provides a sensor value through analog pin A0. Arduino reads this value using analogRead().

5.1 Sensor Reading

The Arduino UNO analog input provides a numerical reading corresponding to the sensor voltage. In the simulation, the value changes according to the light intensity applied to the LDR.

5.2 Threshold-Based Decision

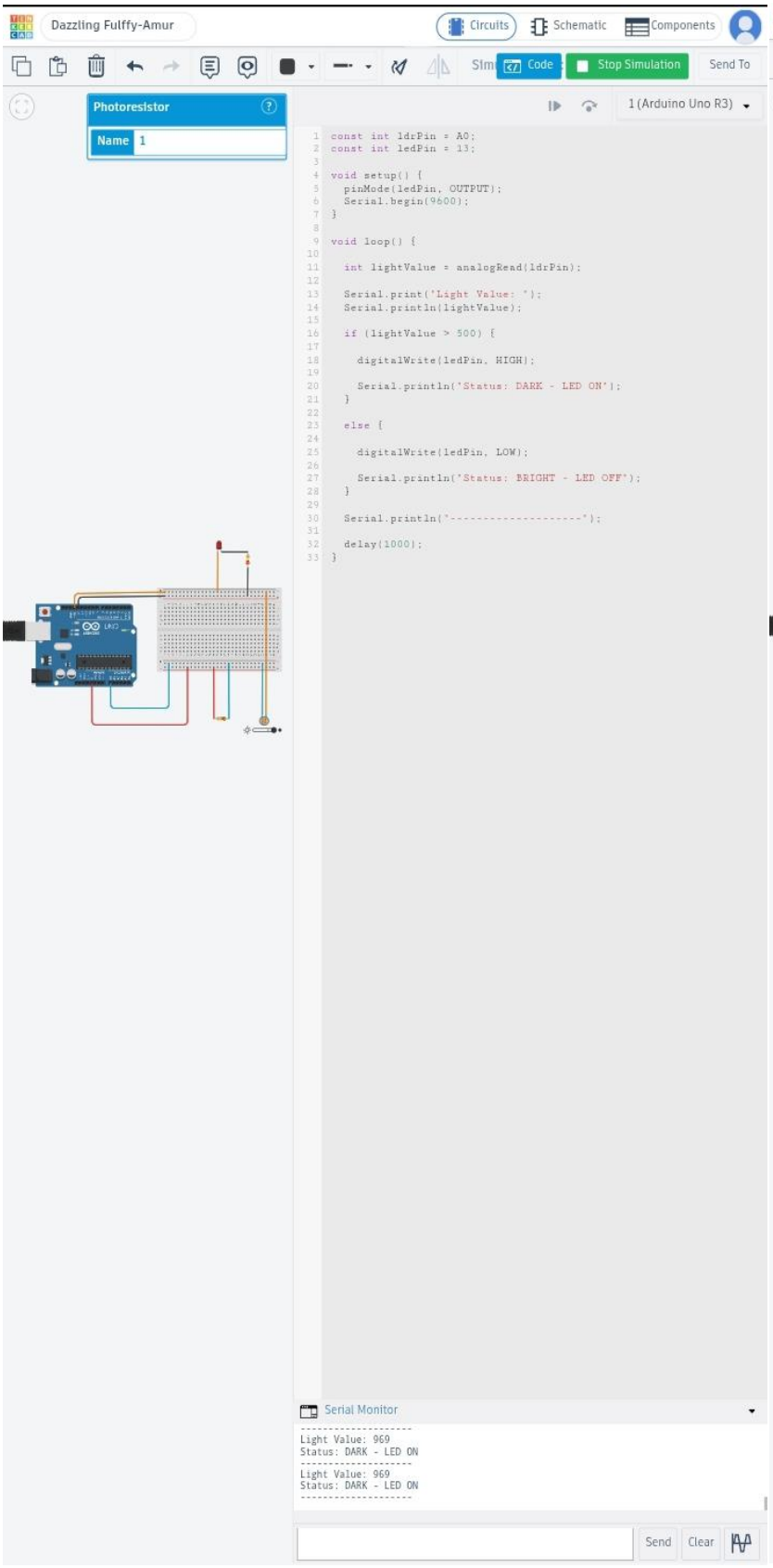
A threshold of 500 is used in the program. If the sensor value is greater than 500, the program interprets the condition as DARK and turns the LED ON. If the value is 500 or below, it interprets the condition as BRIGHT and turns the LED OFF.

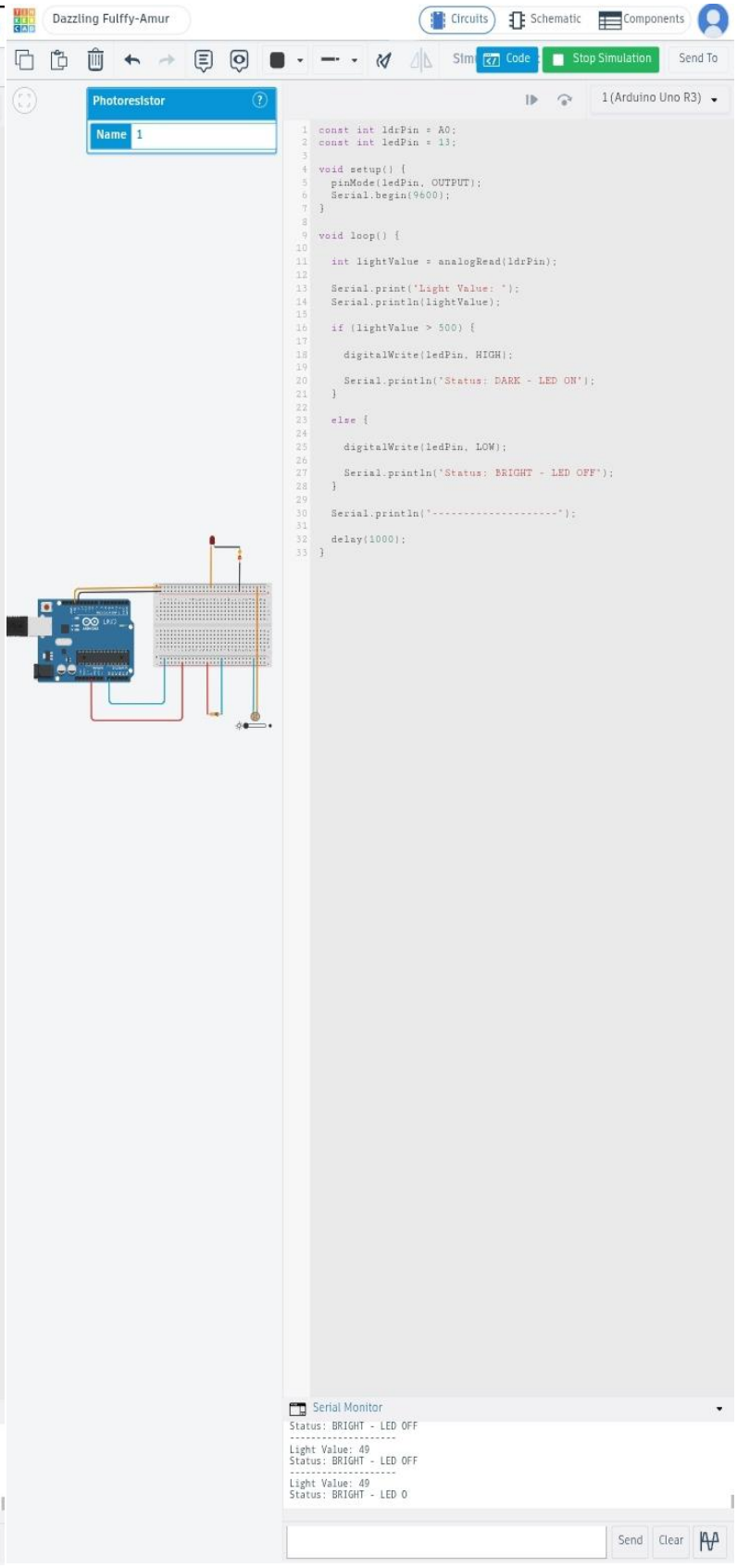
5.3 Automation Logic

```
IF Light Value > 500
    LED = ON
    Status = DARK

ELSE
    LED = OFF
    Status = BRIGHT
```

5.4 Monitoring Logic





After making the decision, Arduino prints the sensor value and the corresponding status to the Serial Monitor. This allows the user to observe the internal decision-making process while the simulation is running.

5.5 Importance of Automation

The project demonstrates the basic automation cycle: sense, process, decide and act. This same concept can be extended to more advanced IoT systems involving Wi-Fi, cloud dashboards, mobile applications and multiple sensors.

6. ARDUINO PROGRAM AND CODE EXPLANATION

6.1 Source Code

```
const int ldrPin = A0;
const int ledPin = 13;

void setup() {
  pinMode(ledPin, OUTPUT);
  Serial.begin(9600);
}

void loop() {
  int lightValue = analogRead(ldrPin);

  Serial.print("Light Value: ");
  Serial.println(lightValue);

  if (lightValue > 500) {
    digitalWrite(ledPin, HIGH);
    Serial.println("Status: DARK - LED ON");
  }
  else {
    digitalWrite(ledPin, LOW);
    Serial.println("Status: BRIGHT - LED OFF");
  }

  Serial.println("--------------------");

  delay(1000);
}
```

6.2 Code Explanation

- `const int ldrPin = A0;` assigns analog pin A0 to the LDR.
- `const int ledPin = 13;` assigns digital pin 13 to the LED.
- `setup()` configures the LED as an output and starts serial communication at 9600 baud.
- `analogRead(ldrPin)` reads the current LDR value.
- `if (lightValue > 500)` checks whether the value is above the selected threshold.
- `digitalWrite(ledPin, HIGH)` turns the LED ON.
- `digitalWrite(ledPin, LOW)` turns the LED OFF.
- `Serial.print()` and `Serial.println()` display readings and status.
- `delay(1000)` creates a one-second interval between readings.

6.3 Programming Approach

The program uses a continuous loop because the sensor must be monitored repeatedly. The simple IF/ELSE structure makes the automation decision easy to understand and modify.

7. SIMULATION, MONITORING AND RESULTS

The project was implemented and tested using Tinkercad Circuits. The simulation allows the LDR light level to be changed and the response of the LED and Serial Monitor to be observed.

7.1 Simulation Procedure

- Build the circuit according to the circuit connections.
- Upload or enter the Arduino source code in the simulator.
- Start the simulation.
- Open the Serial Monitor.
- Change the LDR light intensity.
- Observe the sensor value, LED state and displayed status.
- Repeat the test for both dark and bright conditions.

7.2 Sample Results

Condition	LDR Value	LED	Status
Dark Condition	969	ON	DARK
Bright Condition	49	OFF	BRIGHT

7.3 Serial Monitor Output

```
Dark Condition
Light Value: 969
Status: DARK - LED ON
--------------------

Bright Condition
Light Value: 49
Status: BRIGHT - LED OFF
--------------------
```

7.4 Result Analysis

The simulation demonstrates that the Arduino successfully reads the LDR sensor and makes an automatic decision according to the programmed threshold. When the reading is above 500, the LED turns ON and the Serial Monitor reports a dark condition. When the reading is 500 or below, the LED turns OFF and the Serial Monitor reports a bright condition.

Actual LDR readings may vary depending on the simulated light level. The important result is that the system responds consistently according to the defined threshold.

7.5 Evidence to Include

- Circuit diagram screenshot
- Dark-condition simulation screenshot with LED ON
- Bright-condition simulation screenshot with LED OFF
- Serial Monitor output screenshot

8. APPLICATIONS, ADVANTAGES AND FUTURE SCOPE

8.1 Applications

- Smart home lighting
- Automatic street lighting
- Office and classroom lighting
- Parking-area lighting
- Garden and outdoor lighting
- Energy-saving lighting systems

8.2 Advantages

- Automatic operation without continuous manual control
- Simple and low-cost hardware

- Easy sensor interfacing
- Real-time monitoring through Serial Monitor
- Simple programming and maintenance
- Can be expanded with additional sensors and connectivity

8.3 Limitations

- The current system uses a single sensor and output.
- The threshold is fixed at 500.
- The project is primarily demonstrated through simulation.
- No wireless connectivity is included in the current version.
- Sensor readings can vary with simulation conditions and calibration.

8.4 Future Improvements

- Add an LCD or OLED display for local monitoring.
- Replace Arduino UNO with ESP32 for Wi-Fi/Bluetooth connectivity.
- Send sensor readings to a cloud dashboard.
- Add mobile monitoring and control.
- Use multiple LDRs or additional sensors for larger areas.
- Introduce adjustable or automatically calibrated thresholds.

8.5 IoT Expansion

With an ESP32 or another network-enabled microcontroller, the system can be connected to a cloud platform. Sensor values and LED status could then be monitored remotely. This would convert the basic embedded automation prototype into a more complete IoT-enabled smart lighting solution.

9. CONCLUSION

The Smart Automation System successfully demonstrates how an Arduino UNO can be combined with an LDR sensor to create an automatic, sensor-based control system.

The LDR provides an analog input representing the surrounding light condition. Arduino processes this input using a predefined threshold of 500 and controls the LED automatically. The Serial Monitor provides real-time information about the sensor value and the decision taken by the program.

The project provides practical experience in sensor interfacing, analog data reading, conditional programming, output control and real-time monitoring. It also represents a basic building block for smart home and IoT applications.

The system can be further developed by adding wireless connectivity, displays, cloud monitoring and additional sensors. Therefore, the project provides a useful foundation for understanding embedded automation and future IoT-based systems.

REFERENCES

- Arduino programming concepts and Arduino UNO documentation.
- Tinkercad Circuits simulation environment.
- Embedded Systems and IoT learning resources.
- Project simulation and observations from the implemented Task 3 circuit.